

EFFICIENTNET MODEL

A Comprehensive Research Overview

Topics Covered:
Definition, Architecture & Code

Team
Marwan Ammar
Salma





EfficientNet Research

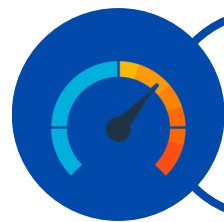
What is EfficientNet?

EfficientNet is a family of convolutional neural networks developed by Google Research in 2019. It introduces a novel compound scaling method that uniformly scales network depth, width, and resolution using a set of fixed scaling coefficients. Unlike traditional approaches that arbitrarily scale only one dimension, EfficientNet balances all three dimensions systematically.

The compound scaling formula uses a compound coefficient ϕ to uniformly scale depth ($d=\alpha^\phi$), width ($w=\beta^\phi$), and resolution ($r=\gamma^\phi$), achieving better accuracy and efficiency.



EfficientNet vs Other Models



EfficientNet

EfficientNet-B0 achieves 77.1% top-1 accuracy on ImageNet with only 5.3M parameters and 0.39B FLOPs.

Uses compound scaling to balance network depth, width, and resolution efficiently.

8.4x smaller and 6.1x faster than ResNet while achieving better accuracy.

Transfer learning ready with ImageNet pre-trained weights available.



Other Models

ResNet-50: 76.0% accuracy with 25.6M parameters. VGG-16: 71.5% accuracy with 138M parameters.

Inception-v3: 78.8% accuracy but requires 23.8M parameters and 5.7B FLOPs.

Traditional models scale only one dimension, leading to diminishing returns.

VGG and ResNet require significantly more computational resources for similar tasks.

Performance Metrics Comparison



EfficientNet Metrics

EfficientNetB0 achieves 77.1% Top-1 accuracy on ImageNet with only 5.3M parameters and 0.39B FLOPS.

EfficientNetB7 reaches 84.3% Top-1 accuracy with 66M parameters.

8.4x smaller than ResNet-50 with higher accuracy and faster inference.

Compound scaling enables optimal accuracy-efficiency trade-off.



Other Models Metrics

ResNet-50 achieves 76.0% Top-1 accuracy but requires 25.6M parameters and 4.1B FLOPS.

VGG-16 achieves only 71.5% accuracy with 138M parameters.

MobileNetV2 has 3.4M params but lower 72% accuracy.

DenseNet-201 uses 20M params for 77% accuracy.



Model Architecture

EfficientNet introduces compound scaling, a method that uniformly scales network depth, width, and resolution using a set of fixed scaling coefficients, achieving better accuracy and efficiency than traditional single-dimension scaling.

Key Building Blocks

- MBConv blocks with squeeze-and-excitation optimization ●
- Depthwise separable convolutions for efficiency ●
- Swish activation function for improved gradients ●



EfficientNet Architecture

EfficientNet introduces compound scaling that uniformly scales network depth, width, and resolution using a compound coefficient. This balanced approach optimizes all three dimensions simultaneously rather than scaling them independently.

Compound Scaling Method

Depth (α): Controls the number of layers in the network

Width (β): Determines the number of channels per layer

Resolution (γ): Sets the input image dimensions



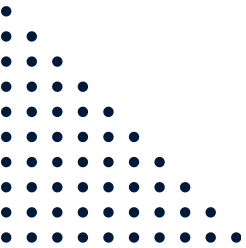
MBConv Blocks

Mobile Inverted Bottleneck Convolution (MBConv) is the core building block of EfficientNet architecture.

Uses depthwise separable convolutions for efficiency

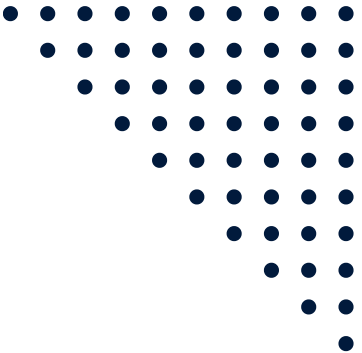
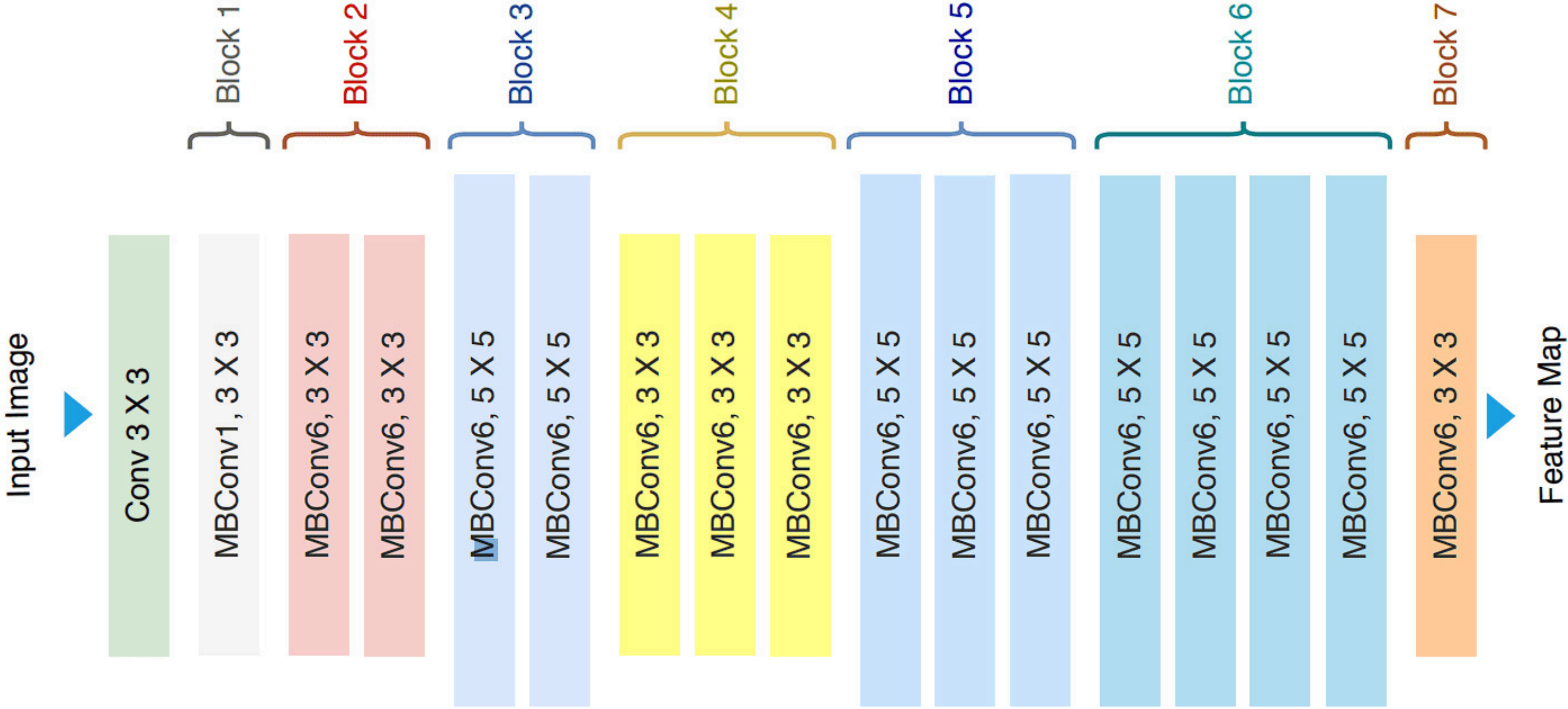
Includes squeeze-and-excitation optimization

Employs inverted residual connections



EfficientNet-B0, discovered through Neural Architectural Search (NAS) is the baseline model. The main components of the architecture **are**:

- MBConv block (Mobile Inverted Bottleneck Convolution)
- Squeeze-and-excitation optimization



Scaling Coefficients

EfficientNet uses compound scaling with coefficient φ to uniformly scale depth (α^φ), width (β^φ), and resolution (γ^φ), where $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$.

Compound Scaling

Depth coefficient $\alpha = 1.2$ controls network layers

Width coefficient $\beta = 1.1$ scales channel numbers

Resolution coefficient $\gamma = 1.15$ adjusts input size



Network Variants

B0: 224×224, 5.3M params

B3: 300×300, 12M params

B7: 600×600, 66M params

MBConv blocks with squeeze-and-excitation

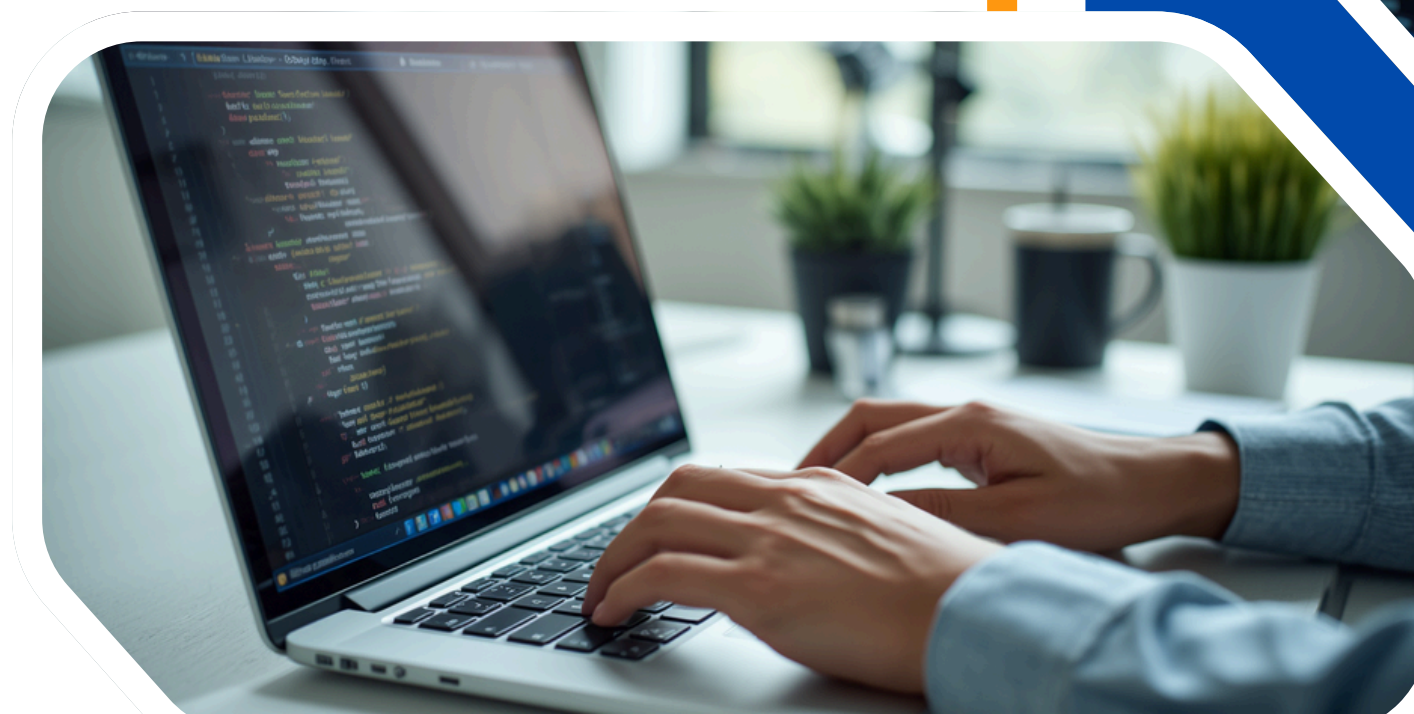
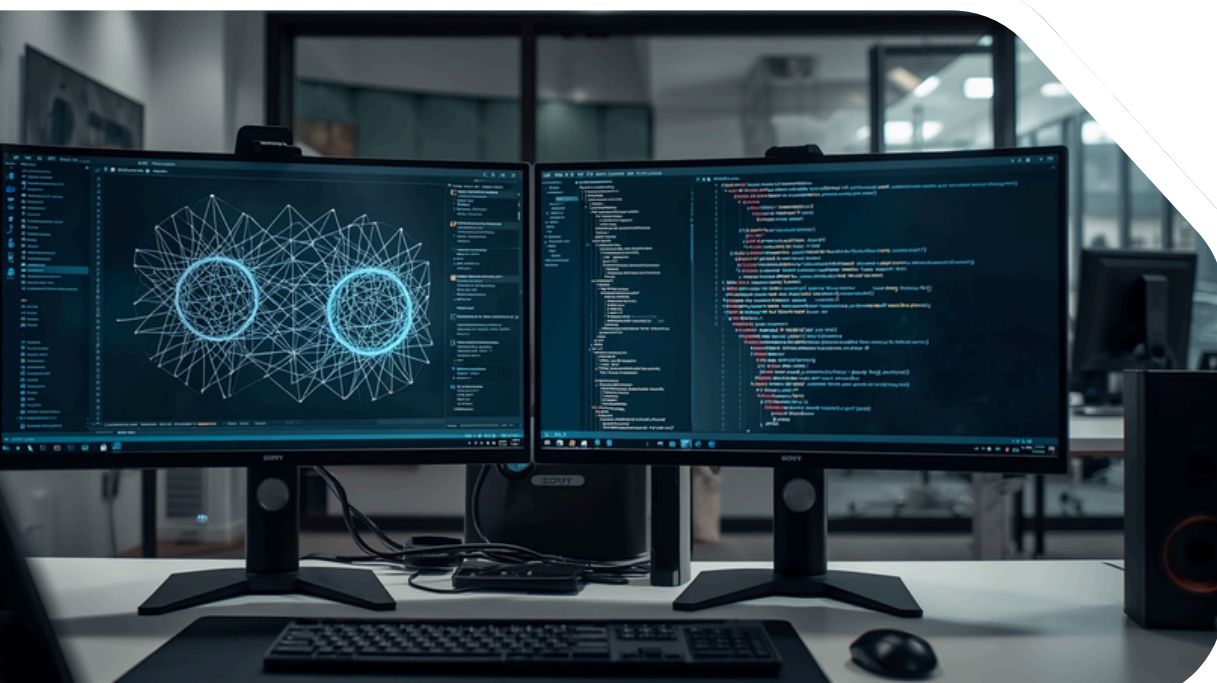
SE ratio of 0.25 for channel attention

Swish activation for improved gradients

Part 3: Implementation

This section presents the practical implementation of EfficientNetB0 using TensorFlow and Keras. The code demonstrates transfer learning by loading pre-trained ImageNet weights, freezing the base model, and adding custom classification layers for a specific task.

Key implementation features include data augmentation, preprocessing, Global Average Pooling, Dropout regularization (0.3), and a Dense layer architecture with softmax output.



Code Breakdown

The implementation starts with EfficientNetB0 as the base model, loaded with pre-trained ImageNet weights. Setting `include_top=False` removes the classification layer, and `trainable=False` freezes all weights for transfer learning. The custom head adds pooling, dropout regularization, and dense layers.

Key Code Components

- Base model frozen (`trainable=False`) to preserve learned features
- Data augmentation pipeline preprocesses inputs before the model
- Custom head: `GlobalAveragePooling2D` → `Dropout` → `Dense` → `Softmax`

PART 1 MODEL BUILDING

Purpose: This loads a pre-trained version of EfficientNetB0 that has already learned to recognize features like edges, textures, and shapes from 1.2 million images (ImageNet).

PART 2 DATA PREPROCESSING

Purpose: This sets up how data enters the model. It adds a "safety net" (Data Augmentation) to prevent overfitting by slightly changing the images (flipping, rotating) and ensures the pixel values are in the specific format EfficientNet expects.

PART 3 CLASSIFIER HEAD

Purpose: The base model output is a complex mathematical map. This chunk simplifies that map into a single prediction. It uses Dropout (turning off random neurons) to ensure the model doesn't rely too heavily on specific pixels, making it more robust.

PART 4 COMPILE

Purpose: This builds the final model object and tells it how to learn. It uses the Adam optimizer (the industry standard) and a loss function suited for multi-class classification where labels are integers.

○○○○

```
# P.1 MODEL BUILDING
base_model = tf.keras.applications.EfficientNetB0(
    include_top=False,
    weights="imagenet",
    input_shape=(224, 224, 3)
)
base_model.trainable = False

# P.2 DATA PREPROCESSING
inputs = tf.keras.Input(shape=(224, 224, 3))
x = data_augmentation(inputs)
x = tf.keras.applications.efficientnet.preprocess_input(x)

# P.3 CLASSIFIER HEAD
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(128, activation="relu")(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(num_classes, activation="softmax")(x)

# P.4 COMPILE & TRAIN
model = tf.keras.Model(inputs, outputs)
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
model.summary()
```



THANK YOU!

Questions?

Key Takeaway:

EfficientNet achieves state-of-the-art accuracy with fewer parameters through compound scaling

